

Sets and Dictionaries

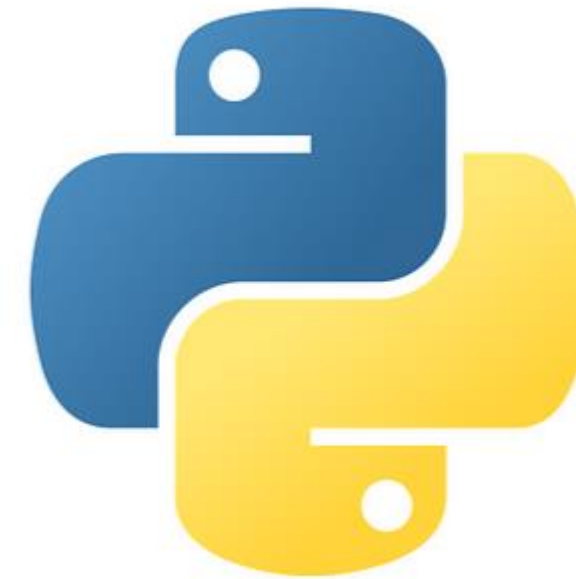


Under Supervisor
ENG: Hazzam Omar

Abdullah Ibrahim Mahmoud

TOPICS

1. Introduction
2. What is a Set & Use Cases of Sets
3. Key Properties & Operations of Sets
4. What is a Dictionary? Use Cases of Dictionaries
5. Key Properties & Operations of Dictionaries
6. Comparison of Sets and Dictionaries
7. Best Practices
8. Conclusion



Introduction

Importance of Data Structures in Python:

Data structures are essential in Python because they allow efficient storage, organization, and manipulation of data. Understanding data structures like lists, tuples, sets, and dictionaries is crucial for writing efficient and effective code.

Focus on Unordered Sequences: Sets and Dictionaries:

This presentation focuses on two specific types of data structures in Python: sets and dictionaries. These are both unordered collections, meaning that the order in which elements are stored or retrieved is not guaranteed. Sets are collections of unique elements, while dictionaries store data as key-value pairs, allowing for fast lookups.

key Concepts: Uniqueness, Hashing, and Efficiency

Uniqueness: Ensures no duplicates

(At dict's value can allow duplication while key not allow)

Hashing: Enables fast data access

(it determines where the data is stored in memory)

Efficiency: Optimized for quick lookups and operations

```
my_set = {1, 2, 3, 3}
print(my_set) # Output: {1, 2, 3}
```

```
my_dict = {'name': 'Alice', 'age': 25, 'age': 30}
print(my_dict) # Output: {'name': 'Alice', 'age': 30}
```

```
my_set = {'apple', 'banana', 'cherry'}
print('banana' in my_set) # Output: True
```

```
my_dict = {'name': 'Alice', 'age': 25}
print(my_dict['name']) # Output: Alice
```

```
items = [1, 2, 2, 3, 4, 4]
unique_items = set(items)
print(unique_items) # Output: {1, 2, 3, 4}
```

What is a Set?

Definition:

- A set is an unordered collection of unique elements.
- Defined using curly braces {} or the set() function.

Example:

```
my_set = {1, 2, 3, 4}
empty_set = set() # Not {}
```

Use Cases of Sets

Practical Applications:

- Removing duplicates from a list.
- Membership testing (fast lookups).
- Mathematical operations on data sets (e.g., finding common elements).

```
myset = {"apple", "banana", "cherry"}
```

Set

Sets are used to store multiple items in a single variable.

Create a Set:

```
thisset = {"apple", "banana", "cherry"}
print(thisset)
```

Duplicates Not Allowed

Duplicate values will be ignored:

```
thisset = {"apple", "banana", "cherry", "apple"}
print(thisset)
```

key Properties & Operations of Sets Package

Properties:

Unordered: No indexing or slicing.

Unique: No duplicates allowed.

Mutable: Can add or remove elements

Operations:

- Add ()
- Remove ()
- pop ()
- clear ()
- update ()

common Operations:

Union (| or union()): Combines elements from both sets.

Intersection (& or intersection()): Elements common to both sets.

Difference (- or difference()): Elements in one set but not the other.

Symmetric Difference (^ or symmetric_difference()): Elements in either set but not in both.

```
# initialize my_set
my_set = {1, 3}
print(my_set)
my_set.add(2)
my_set.add(4)
print(my_set)
my_set.remove(2)
print(my_set)
```

```
# This will raise a TypeError
invalid_set = {1, 2, [3, 4]} # TypeError: unhashable type: 'list'
print(my_set)
# So, while the set itself is mutable
# its elements must be immutable.
# To solve this is doing
my_set.update([4, 5], {1, 6, 8})
print(my_set)
# important
# The method itself does not return an error even if the iterable is mutable,
# because it just adds the elements from the provided iterables into the set.
# i can modify in this
# output from merge between list and set but in unique
```

```
# Symmetric difference of two sets
# initialize A and B
A = {1, 2, 3, 4, 5}
B = {4, 5, 6, 7, 8}

# use ^ operator
# Output: {1, 2, 3, 6, 7, 8}
print(A ^ B) # all in a not in b and (union) all in b not in a
print(A.symmetric_difference(B))
```

```
# Set union method
# initialize A and B
A = {1, 2, 3, 4, 5}
B = {4, 5, 6, 7, 8}

# use | operator UNION
# Output: {1, 2, 3, 4, 5, 6, 7, 8}
print(A | B) # A U B # pipe
print(A.union(B)) # onther method
print(B.union(A))
```

```
# Intersection of sets
# initialize A and B
A = {1, 2, 3, 4, 5}
B = {4, 5, 6, 7, 8}

# use & operator INTERSECTION
# Output: {4, 5}
print(A & B)
print(A.intersection(B))
print(B.intersection(A))
```

```
# Difference of two sets
# initialize A and B
A = {1, 2, 3, 4, 5}
B = {4, 5, 6, 7, 8}

# use - operator on A
# Output: {1, 2, 3}
print(A - B)
print(A.difference(B)) # all in a not in b
# !=
print(B - A)
print(B.difference(A))
```

What is a Dictionary?

Use Cases of Dictionaries

Definition:

- A dictionary is an unordered collection of key-value pairs.
- Defined using curly braces {} with key-value pairs separated by colons.

Example

```
my_dict = {"name": "Alice", "age": 25}
empty_dict = {}
```

```
# A dictionary is a collection of a key-value pairs.
human1 = {'height': 180, 'weight': 80, 'eye-color': 'green', 'language': 'mongolian'}
```

Use Cases of Dictionaries

Practical Applications:

- Storing and retrieving data by key.
- Counting occurrences of elements.
- Creating mappings (e.g., word frequencies, lookup tables).

```
# another method to define dict by lines
human2 = {'height': 160,
          'weight': 52,
          'eye-color': 'brown',
          'language': 'british',
          'sex': 'female'}
human2
```

Key Properties & Operations of Dictionaries

Properties:

- Unordered: No indexing .
- Keys are unique and immutable (e.g., strings, numbers).
Values can be of any data type.

Operations:

- Get ()
- Setdefault ()
- Keys ()
- Values ()
- Items ()

Common Operations:

- Adding/Updating:

my_dict["key"] = value

- Removing:

del my_dict["key"], pop

Accessing:

my_dict["key"], get()

```
my_dict = {}
my_dict["age"] = 25
my_dict["city"] = "Cairo"
age = my_dict.get("age", 0) # Returns 25
print(age)
print(my_dict)
my_dict.pop("age")
my_dict
```

Comparison of Sets and Dictionaries

Similarities:

- Both are unordered collections.
- Both use hashing for fast lookups.

Differences:

- Sets store unique elements, while dictionaries store key-value pairs.
- Dictionaries are more versatile for storing complex data.

Best Practices

Use Sets:

- When you need unique elements.
- For membership testing.

Use Dictionaries:

- When you need to associate keys with values.
- For fast data retrieval by key.

Conclusion

Summary:

- Sets and dictionaries are essential tools in Python for handling unordered collections.
- Sets ensure uniqueness, while dictionaries pair keys with values.
- Efficient data manipulation and retrieval.

